46. API Gateways and Edge Patterns

46. API Gateways and Edge Patterns

Phase: Phase 10 — Load balancing and proxies
Estimated time: 1–2 days
Slug: `46-api-gateways`

Learning objectives

- Place auth/rate-limit/WAF at edge
- Distinguish gateway vs service mesh
- Debug 502/503/504 meanings
- Trace request IDs across hops

Prerequisites

Complete prior lessons. Prefer a Linux VM for `ip`/`ss`/`nft` examples; macOS works for many `curl`/`dig` drills. Type every command yourself and keep a lab notes file.

Why this matters

Mobile apps, browsers, and partners share one edge. API gateways add auth, rate limit, and routing — and new failure domains at the edge.

Core concepts

What you must be able to explain:

- Place auth/rate-limit/WAF at edge
- Distinguish gateway vs service mesh
- Debug 502/503/504 meanings
- Trace request IDs across hops

Mental model for `46-api-gateways` (exercise kind `gw_504`):

- Prefer **measurable** checks (exit codes, ICMP replies, DNS rcode, HTTP status, socket state) over vibes.
- Separate **reachability** (can packets flow?) from **correctness** (right service, cert, route, identity).
- Ask: *which layer's tool falsifies my current hypothesis?* Then run that tool.
- Record the exact command that proved the root cause — that becomes your playbook snippet.

Worked commands

Run these on a lab host. Linux VM preferred for `ip`/`ss`/`tcpdump`/`nft`; macOS is fine for many `curl`/`dig`/`ping` drills. Your addresses and timings will differ — match the *shape* of the output and the conclusion you draw.

Example 1

```
ip -br addr; ip route; ip route get 1.1.1.1
```

Sample output:

```
eth0 UP 192.168.1.20/24
default via 192.168.1.1 dev eth0
1.1.1.1 via 192.168.1.1 dev eth0 src 192.168.1.20
```

Interpretation: `ip route get` shows the chosen route, egress device, and preferred source — gold for 'why did it leave this NIC?' questions.

Example 2

```
ping -c 3 1.1.1.1
ss -tulpn | head
# macOS: netstat -anv | head
```

Sample output:

```
3 packets transmitted, 3 received
udp ... :53
tcp LISTEN ... :22
```

Interpretation: ICMP OK proves basic L3; ss proves local listeners. Combine with ARP/neigh when LAN peers fail.

Example 3

```
ip neigh show | head
ping -c 1 $(ip route | awk '/default/ {print $3; exit}')
```

Sample output:

```
192.168.1.1 dev eth0 lladdr ... REACHABLE
64 bytes from gateway
```

Interpretation: Neighbor/ARP state validates L2 adjacency to the gateway before you blame the wider internet.

Mini-lab

1. Re-type every worked command for **API Gateways and Edge Patterns** and paste outputs into lab notes.
2. Introduce one controlled failure in the lab (wrong DNS, firewall drop, bad route) and recover using only this lesson's tools.
3. Write a 5-line playbook: symptom → first command → expected healthy output.
4. Complete the exercise stubs; then explain each function's return value without looking.

Failure modes

Symptom	Likely cause	Next check
Business logic only in gateway forever	See common mistakes	Re-run lesson commands
No correlation IDs	See common mistakes	Re-run lesson commands
Retry storms without backoff	See common mistakes	Re-run lesson commands
Timeout with no RST	Silent drop / blackhole	tcpdump + compare path

Study tip for API Gateways and Edge Patterns

Before each new command, write a one-line hypothesis. If the result falsifies it, update the hypothesis instead of stacking random tools. Note equivalents across OS (`ip` ↔ `ifconfig`/`route`, `ss` ↔ `netstat`). Re-run two commands from this lesson tomorrow from memory and explain the sample outputs out loud.

Evidence habit

When pages are down, prefer evidence (`ip route get`, resolver answer, packet capture, listener list) over intuition. Document **command → output → conclusion** in a lab notebook so the same path is reusable on-call. Change one variable at a time; write a one-line hypothesis before each new tool.

Common mistakes

- Business logic only in gateway forever
- No correlation IDs
- Retry storms without backoff

Exercises

Work in `exercises/46-api-gateways/`. Read that folder's `README.md` before coding.

```
cd exercises/46-api-gateways
chmod +x test.sh exercise.sh 2>/dev/null || true
./test.sh
```

What you will build

Edit `exercise.sh` and implement `http_504_often`, `http_502_often`, `edge_pattern`. Each function is a tiny retrieval drill for **API Gateways and Edge Patterns** — return the exact tokens/flags/labels from the lesson. Keep the comment above each function describing when you would use that fact on-call. Run `./test.sh` until all 3 assertions pass.

Study loop

- **A (Drill):** make tests pass
- **B (Apply):** run 2–3 real commands from this lesson in your lab and note outputs
- **C (Challenge):** rewrite answers from memory next day

Do not open `solutions/46-api-gateways/` until after an honest attempt.

Self-check

Answer without scrolling up:

1. 504 often means?
2. 502 hints?
3. Why request IDs?

Further reading

- https://www.rfc-editor.org/rfc/rfc9110